

Software Architecture and Design Specification

Project: Chat Application using Socket Programming

Authors: Sriya Yashita Chandu; Abhishek B Khanapur; Thai Rishi Saran Datta; Sukriti Deep

Date: 06-10-2026

Revision History

Version | Date | Author | Change Summary

Approvals

Role | Name | Signature/Date

1. Introduction

1.1 Purpose

This document describes how the chat application is built: the main parts of the system, how they talk to each other, the technology we are using and the risks we have planned for. It is based on the requirements in our SRS. Coding has not started yet, so this is the design the implementation will follow. If the code ever needs to differ, we update this document first.

1.2 Scope

It covers both the desktop client and the central server: signing up and logging in, secure connections, seeing who is online, chat rooms, sending and receiving messages in real time, rejecting bad input, saving chat history, join and leave notifications, dealing with clients that drop off unexpectedly, admin tools, logging and session handling. Voice and video calls, payments and social features are out of scope.

1.3 Audience

Mainly the developers building the client and server, and the testers writing test cases against it. It is also meant for our project guide, the course coordinator, anyone checking that the design covers the requirements, and whoever ends up running the server.

1.4 Definitions

Terms already defined in the SRS (socket, client, server, chat room, session, TCP, FR, NFR, RTM, RBAC) mean the same thing here. A few extra terms used in this document:

Term	Definition
ADR	Architecture Decision Record: a short write-up of a design decision, what we considered and why we chose it.

Term	Definition
Component	A self-contained part of the system with one clear job.
Frame	One message as it travels over the network: a 4-byte length followed by a JSON body.
Outbound queue	Each connection's own line of messages waiting to be sent to that client.
Session token	A random value the server hands out after login to recognise that user's connection.
TLS	Transport Layer Security; encrypts the connection between client and server.
Certificate pinning	The client only trusts the exact server certificate it was shipped with, so nobody can sit in the middle with a fake one.
scrypt	A slow-by-design password hashing function built into Python, used with a random salt for each user.
WAL	A SQLite mode that lets the database be read while something is being written.
STRIDE	A checklist of threat types: spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege.
PlantUML	The tool we used to draw the diagrams.

2. Document Overview

2.1 How to use this document

The architecture part explains what the system is made of and why we chose this structure. The design part goes a level deeper into the messages the client and server exchange and how a request moves through the system. Requirement IDs such as CHAT-F-001 come straight from the SRS, and the traceability table shows where each one is handled and which test checks it.

2.2 Related Documents

Document	Version / Date	Relevance
Software Requirements Specification (SRS)	v1.0, 04-09-2026	Where all the requirements come from
Software Test Plan (STP)	v1.0, 28-09-2026	Test case IDs used in the traceability table
Requirements Traceability Matrix (RTM)	Part of the SRS	Original requirement-to-module mapping
Source repository	To be linked	Code layout follows the module layout in this document

3. Architecture

3.1 Goals & Constraints

Each goal comes from a measurable requirement in the SRS, so we can actually test whether we hit it.

Goal	Target	Source
Low-latency delivery	90% of valid messages delivered to connected recipients within 1 s under normal load	CHAT-NF-001
Concurrency	At least 50 simultaneous authenticated clients without critical failure	CHAT-NF-002
Fault isolation	One client's disconnect or error never terminates other sessions	CHAT-NF-003, CHAT-F-014
Confidentiality	All client-server traffic encrypted; no plaintext passwords stored or logged	CHAT-NF-005, SR-001, SR-002
Robust input handling	Malformed or oversized input rejected without affecting the server	CHAT-F-009, CHAT-SR-004
Data integrity	No malformed or orphan records in persistent storage	CHAT-NF-006
Maintainability	Socket handling, authentication, routing, persistence and UI independently maintainable	CHAT-NF-007

Constraints

- Communication runs over TCP sockets, with TLS on top for encryption.
- Server host, port, message size and room limits must be configurable.
- Everything is written in Python 3.11 or later and runs on Windows, Linux or macOS.
- Chat history is saved, so storage and data integrity are part of the design.
- There is one central server with no clustering; we accept this as a known limitation.
- No paid services or extra infrastructure; we stick to Python's standard library wherever we can.

3.2 Stakeholders & Concerns

Stakeholder	Primary concerns	Addressed by
User	Fast, reliable delivery; privacy of messages and credentials; clear connection status	Message Router, TLS transport, Client UI status bar
Administrator	Ability to restrict users and manage rooms; visibility of abuse	Admin Service, Audit Logger
Server Operator	Server stays up when clients misbehave; configurable deployment; usable logs	Connection Manager isolation, config file, rotated logs
Development team	Clear module boundaries, testable units, simple dependencies	Layered structure and a clear module layout
Testers	Every requirement verifiable and traceable	Traceability table and clear error codes
Project guide / evaluators	Consistency with SRS and STP; justified decisions	Documented design decisions and traceability

3.3 Component (UML) Diagram

The server is split into four layers. A request comes in through the transport layer, gets checked by the protocol layer, is handled by one of the services, and anything that needs

saving goes through the persistence layer. Replies and notifications go back out through each client's own outgoing queue, and anything security-related is written to the audit log.

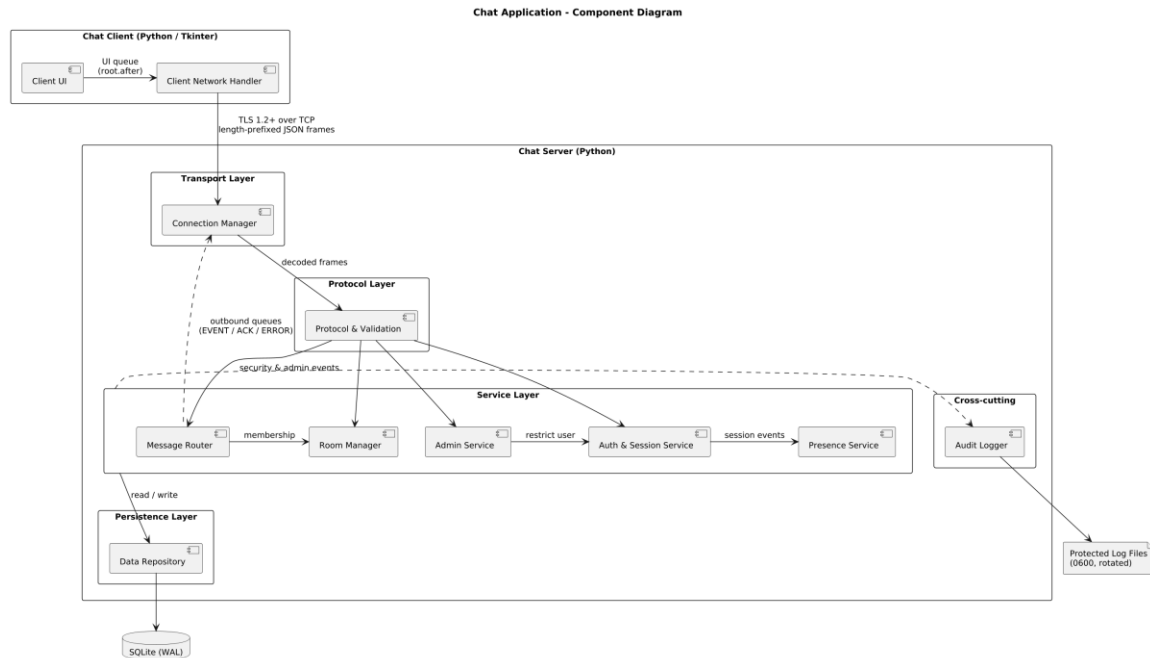


Figure 1. Component diagram of the chat client and server

3.4 Component Descriptions

Component	Layer	Responsibility	Req.
Client UI	Client	The Tkinter windows the user sees: sign-up and login, room list, who is online, the chat itself, history and a status bar. It only updates from the main thread and picks up network events from a queue every 50 ms. Your own messages look different from everyone else's.	F-004, NF-004
Client Network Handler	Client	Opens the encrypted connection, checks it is talking to the real server, packs and unpacks messages, listens in the background, sends a heartbeat every 30 s and tells the UI if the connection drops.	F-003, F-018, SR-001
Connection Manager	Transport	Accepts new connections and limits how many can be open at once. Each client gets one thread reading from it and one writing to it, plus its own outgoing queue. When a client disconnects or errors out, it cleans up that client only, without touching anyone else.	F-003, F-014, NF-002, NF-003
Protocol & Validation	Protocol	Reads each message's length first and throws away anything too big before reading the rest. Checks the JSON, the message type and every field. Before login it only allows register, login and ping. Valid requests are passed to the right service.	F-009, SR-004
Auth & Session Service	Service	Handles sign-up (unique usernames, hashed passwords), login, blocked accounts, session tokens and logout. A background thread closes sessions that have been idle too long.	F-001, F-002, F-015, F-018, SR-002, SR-006
Presence Service	Service	Keeps track of who is online and lets everyone know when someone logs in, logs out or drops off.	F-004

Component	Layer	Responsibility	Req.
Room Manager	Service	Creates rooms with unique names, handles joining and leaving, checks permissions and announces when people come and go.	F-005, F-006, F-013
Message Router	Service	Makes sure the sender is allowed to post, timestamps the message, sends it off to be saved, delivers it to everyone who should get it and tells the sender whether it went through. Also serves chat history.	F-007, F-008, F-010, F-012
Admin Service	Service	Admin-only actions: activating, deactivating or restricting users (and kicking them off if they are online), and creating, renaming, restricting or removing rooms. Every action checks the user really is an admin.	F-015, F-016, SR-003
Data Repository	Persistence	The only part of the system that touches the database. All writes go through one thread to avoid conflicts, every query is parameterised, and foreign keys are enforced.	F-011, F-012, NF-006
Audit Logger	Cross-cutting	Records logins, admin actions, errors and other important events with who, when and what happened. Passwords and tokens are stripped out before anything is written, and only the server can write to the log files.	F-017, SR-002, SR-005
SQLite database	Data store	Stores users, rooms, room members, messages, sessions and the audit trail.	F-011, NF-006

The SRS lists a few module names (such as MessageModule, DeliveryModule and NotificationModule) that we have merged into the components above. The traceability table shows which is which.

3.5 Chosen Architecture Pattern and Rationale

Decision record: client-server, with a layered server and one thread pair per connection

Status: Accepted

Context. We need one central place to handle logins, message routing, moderation and logging. The system also has to handle at least 50 users at once, deliver messages within a second, keep one client's crash from affecting anyone else, and stay easy to maintain as separate pieces. We are building it in Python from scratch.

Decision.

- **Overall style:** client-server. Every client connects to one central server, which holds all the state and enforces the security rules.
- **Server structure:** four layers: transport (connections), protocol (reading and checking messages), services (login, presence, rooms, routing, admin) and persistence (the database). Each layer only talks to the one directly below it.
- **Handling many users:** each connection gets one thread for reading and one for writing, with its own outgoing queue, and there is a cap on total connections (100 by default).

Alternatives considered.

Alternative	Reason rejected
Peer-to-peer messaging	No central place for logins, moderation or logging, which the SRS needs.
One shared lock for sending instead of a writer per client	One slow client would hold up messages for everyone.
asyncio event loop	More than we need for 50 users, and the database, password hashing and Tkinter all block, so we would be working around asyncio constantly.
Microservices	Far too much setup and overhead for a single-server project.

Consequences.

- Each layer can be tested on its own by faking the layer below it.
- Because only one thread writes to each connection, messages never get mixed together, and each sender's messages stay in order.
- Threads grow with the number of users (about 100 at 50 users), which is fine for this kind of work, and the connection cap stops it getting out of hand.
- If the one server goes down, chat goes down. For a project of this size we accept that.

Supporting decisions.

Concern	Decision	Rationale
Message framing	4-byte big-endian length prefix + UTF-8 JSON body; max frame 8 KB	TCP is just a stream of bytes, so we need to mark where each message ends. Reading the length first lets us reject oversized messages before reading them.
Database	SQLite in WAL mode; single writer thread	It is relational as the SRS asks, needs no separate database server, and one writer avoids locking problems.
Persistence timing	Forward first, write asynchronously	Saving to disk never slows down delivery.
Password storage	hashlib.scrypt, 16-byte random salt per user	A strong one-way hash that comes with Python, so no extra library.
Transport security	TLS 1.2+, self-signed server certificate pinned in client	Encrypts all traffic. Pinning matters because a self-signed certificate on its own would not stop someone intercepting the connection.
Client threading	Network thread passes events to UI through a queue	Tkinter crashes if you update it from another thread.

3.6 Technology Stack & Data Stores

Concern	Technology	Notes
Language / runtime	Python 3.11+	Same runtime for client and server
Networking	socket, ssl, struct	TLS 1.2 minimum; length-prefix framing
Concurrency	threading, queue	Reader/writer thread per connection; DB writer thread; session sweeper
Serialisation	json	UTF-8 JSON message bodies

Concern	Technology	Notes
Security	hashlib.scrypt, hmac.compare_digest, secrets	Password hashing, constant-time comparison, session tokens
Persistence	sqlite3 (SQLite, WAL mode)	Parameterised queries only; foreign keys ON
Logging	logging, RotatingFileHandler	Redaction filter; 0600 file permissions
Client UI	tkinter	Bundled with Python
Testing	pytest	Only third-party dependency
Diagrams	PlantUML	Sources kept in the repository docs folder

Apart from pytest, everything comes from Python's standard library, so there is nothing extra to install and fewer outside dependencies to worry about.

Data store (SQLite)

Table	Key fields	Integrity rules
User	UserID (PK), Username (UNIQUE), PasswordHash, Salt, Role, Status, CreatedAt	Role ∈ {USER, ADMIN}; Status ∈ {ACTIVE, RESTRICTED, DEACTIVATED}
ChatRoom	RoomID (PK), RoomName (UNIQUE), CreatedBy (FK User), Status	Status ∈ {OPEN, RESTRICTED, REMOVED}
RoomMember	RoomID (FK), UserID (FK), JoinedAt, Status	Composite PK (RoomID, UserID)
Message	MessageID (PK), SenderID (FK), RoomID (FK, nullable), RecipientID (FK, nullable), Content, SenderSeq, Timestamp	Exactly one of RoomID / RecipientID set; Content length ≤ max_text_chars
Session	SessionID (PK), UserID (FK), TokenHash, ConnectedAt, LastActivity, DisconnectedAt	Only a hash of the token is stored
AuditLog	AuditID (PK), UserID (FK, nullable), Action, Entity, Outcome, Timestamp	Insert-only from the server; no update/delete path

Configuration settings

Setting	Default	What it controls
host / port	0.0.0.0 / 5050	Where the server listens
max_frame_bytes	8192	Largest message the server will accept
max_text_chars	2000	Longest chat message text
max_connections	100	How many clients can be connected at once
max_rooms	50	How many rooms can exist
idle_timeout_seconds	900	How long before an idle session is closed
heartbeat_seconds	30	How often client and server check the connection is alive
tls_cert_path / tls_key_path	certs/server.crt, certs/server.key	Server certificate and private key
db_path / log_path	data/chat.db, logs/server.log	Where the database and logs are stored

3.7 Risks & Mitigations

ID	Risk	Impact	Mitigation	Req.
R1	Slow or stalled recipient blocks fan-out	Latency target missed for all users	Per-client outbound queue and writer thread; client dropped if its queue exceeds a bound	NF-001
R2	Thread exhaustion under many connections	Server stops accepting clients	max_connections cap; excess connections get ERROR SERVER_BUSY and are closed	NF-002
R3	Partial reads on the TCP stream corrupt framing	Garbled or merged messages, crashes	recv_exact() reads exactly N bytes; malformed-frame and fuzz tests in the unit suite	F-009, SR-004
R4	Crashed clients leave ghost sessions	Wrong presence list, leaked resources	PING/PONG heartbeat, socket timeouts, idle-session sweeper	F-014, SR-006
R5	Database writes or locking delay delivery	Latency target missed	Forward-then-persist; single DB writer thread; WAL mode	NF-001, NF-006
R6	Passwords or tokens appear in logs	Credential exposure	Redaction filter on all handlers; TC-LOG-01 and TC-SEC-03 inspect logs	SR-002, F-017
R7	Self-signed certificate accepted without verification	Traffic can be intercepted	Client pins the server certificate and refuses any other	SR-001
R8	Tkinter updated from the network thread	Random client crashes	All UI updates go through a queue drained on the Tk main thread	NF-004
R9	Login errors reveal whether a username exists	Account enumeration	Same AUTH_FAILED code for wrong password and restricted account; reason logged only	SR-003, F-015
R10	Single central server	Server outage stops all chat	Accepted for a project of this size; the server restarts quickly and picks up its data from the database	NF-003
R11	Implementation not yet started	Schedule pressure on high-priority features	Build in the order of the sequence diagrams: connecting and logging in first, then messaging, then rooms, history and admin	High-priority FRs

3.8 Traceability to Requirements

This table links every requirement in the SRS to the part of the system that handles it and the test that checks it. We have kept the module name the SRS uses alongside ours so the two documents line up.

Req ID	Requirement	SRS module name	Handled by	Test case
CHAT-F-001	User registration	AuthModule	Auth & Session Service; Data Repository	TC-REG-01
CHAT-F-002	Authentication	AuthModule	Auth & Session Service; Protocol & Validation	TC-AUTH-01
CHAT-F-003	Socket connection	SocketModule	Connection Manager; Client Network Handler	TC-CONN-01
CHAT-F-004	Status / online users	PresenceModule	Presence Service; Client UI	TC-USER-01
CHAT-F-005	Create room	RoomModule	Room Manager	TC-ROOM-01

Req ID	Requirement	SRS module name	Handled by	Test case
CHAT-F-006	Join / leave room	RoomModule	Room Manager	TC-ROOM-02
CHAT-F-007	Send message	MessageModule	Message Router	TC-MSG-01
CHAT-F-008	Ordered delivery	MessageRouter	Message Router; Connection Manager	TC-MSG-02
CHAT-F-009	Reject invalid / oversized	ValidationModule	Protocol & Validation	TC-MSG-03
CHAT-F-010	Unavailable recipient	DeliveryModule	Message Router	TC-MSG-04
CHAT-F-011	Store messages	HistoryModule	Data Repository	TC-HIST-01
CHAT-F-012	Retrieve history	HistoryModule	Message Router; Data Repository	TC-HIST-02
CHAT-F-013	Join / leave notifications	NotificationModule	Room Manager	TC-NOTIF-01
CHAT-F-014	Unexpected disconnection	ConnectionManager	Connection Manager; Auth & Session Service	TC-DISC-01
CHAT-F-015	Admin account control	AdminModule	Admin Service; Auth & Session Service	TC-ADMIN-01
CHAT-F-016	Admin room management	AdminModule	Admin Service; Room Manager	TC-ADMIN-02
CHAT-F-017	Protected logging	LoggingModule	Audit Logger	TC-LOG-01
CHAT-F-018	Logout	SessionModule	Auth & Session Service; Connection Manager	TC-LOGOUT-01
CHAT-NF-001	Latency ≤ 1 s (p90)	PerformanceModule*	Decision: outbound queues, async persistence	TC-PERF-01
CHAT-NF-002	50 concurrent clients	ScalabilityModule*	Decision: thread-per-connection + cap	TC-LOAD-01
CHAT-NF-003	Failure isolation	ReliabilityModule*	Connection Manager	TC-REL-01
CHAT-NF-004	Usability	UsabilityModule*	Client UI	TC-UX-01
CHAT-NF-005	Encryption, no plaintext secrets	Security/PrivacyModule*	Connection Manager; Auth & Session; Audit Logger	TC-SEC-01
CHAT-NF-006	Data integrity	Data IntegrityModule*	Data Repository	TC-DATA-01
CHAT-NF-007	Modularity	MaintainabilityModule*	Decision: layered architecture	TC-MAINT-01
CHAT-SR-001	Encrypted transport	SecurityModule	Connection Manager; Client Network Handler	TC-SEC-02
CHAT-SR-002	Salted password hashes	SecurityModule	Auth & Session Service; Audit Logger	TC-SEC-03
CHAT-SR-003	Role-based authorisation	SecurityModule	Admin Service; Room Manager; Message Router	TC-SEC-04
CHAT-SR-004	Input validation	SecurityModule	Protocol & Validation	TC-SEC-05
CHAT-SR-005	Protected audit logs	SecurityModule	Audit Logger	TC-SEC-06
CHAT-SR-006	Session expiry	SecurityModule	Auth & Session Service	TC-SEC-07

** These are quality requirements rather than pieces of code, so they are met by design choices (such as the per-client queues and the thread model) instead of a single component.*

3.9 Security Architecture

3.9.1 Security Objectives

The security architecture protects the confidentiality, integrity and availability of the chat application. Security controls are implemented across the client, network, server services and persistence layers.

The main security objectives are:

- Protect client-server communication from interception.
- Protect user passwords from disclosure.
- Prevent unauthorized access to rooms, messages and administrative functions.
- Reject malformed or oversized client input.
- Protect security and audit logs from unauthorized modification.
- Prevent the reuse of expired or invalid sessions.
- Prevent one malicious or faulty client from affecting other active sessions.

3.9.2 Threat Model

The system uses a STRIDE-based threat model to identify threats at the client, transport, service and persistence layers. The main assets are user credentials, session tokens, chat messages, account and room information, and audit records.

Asset / Area	Threat	Required Protection
Authentication	Credential theft or account impersonation	Password hashing, authentication checks and session security
Network communication	Interception or modification of traffic	TLS and server certificate verification/pinning
Chat messages	Unauthorized access or malformed input	Session validation, authorization and input validation
Administration	Unauthorized privileged actions	Role-based authorization
Logs	Exposure or modification of security records	Redaction and protected server-side logging

Sessions	Reuse of stale or invalid sessions	Session tokens, timeout and invalidation
----------	------------------------------------	--

3.9.3 STRIDE Analysis

Threat	Potential Attack	Security Control
Spoofing	Attacker attempts to impersonate a user or server.	User authentication, session tokens, TLS and certificate pinning
Tampering	Attacker attempts to modify messages during transmission.	TLS-protected communication and server-side validation
Repudiation	A user denies performing a security or administrative action.	Protected audit logging with user, action, outcome and timestamp
Information Disclosure	Credentials, session information or chat messages are exposed.	TLS encryption, password hashing and log redaction
Denial of Service	Malformed/oversized input or excessive connections affect the server.	Input limits, validation and maximum connection limits
Elevation of Privilege	A normal user attempts administrative operations.	RBAC and server-side authorization checks

3.9.4 Authentication

Authentication is performed by the server. Before successful authentication, an unauthenticated connection is permitted to perform only REGISTER, LOGIN and PING operations. After successful login, the server creates an authenticated session and provides a session token to the client.

User passwords are never stored in plaintext. The design uses hashlib.scrypt with a random 16-byte salt for each user. Password comparisons use hmac.compare_digest. Only a hash of the session token is stored in the database.

3.9.5 Authorization / RBAC

The server enforces Role-Based Access Control (RBAC). The two defined roles are USER and ADMIN. Normal users can perform permitted chat operations, while administrative operations are restricted to authenticated administrators.

Every administrative request is checked by the server before execution. Unauthorized protected operations return the FORBIDDEN error code.

3.9.6 Secure Communication

Each client communicates with the server over a TCP connection protected using TLS 1.2 or higher. The client verifies the configured server certificate using certificate pinning, preventing an unexpected certificate from being accepted. This protects credentials, session information and chat messages while in transit.

3.9.7 Input Validation

All client input passes through the Protocol & Validation component before reaching application services. The server validates the frame length, JSON format, message type, required fields, field values and message size.

The default maximum frame size is 8192 bytes and the maximum chat text length is 2000 characters. Malformed, invalid or oversized requests are safely rejected.

3.9.8 Protected Logging

The Audit Logger records important security and operational events such as login attempts, administrative actions and errors. Passwords and session tokens are removed before data is written to logs. Only the server can write to the log files, and audit records are insert-only.

Log files use a redaction filter and restricted file permissions. Security logs therefore provide traceability without exposing sensitive credentials or tokens.

3.9.9 Session Security

Authenticated sessions are invalidated when the user logs out, the configured idle timeout is reached, the connection is lost, a heartbeat fails, or an administrator restricts the account. The default idle timeout is 900 seconds and the heartbeat interval is 30 seconds.

The server returns SESSION_EXPIRED when a session token is missing, invalid or inactive beyond the configured timeout.

4. Design

4.1 Design Overview

The client and server talk over a single encrypted TCP connection per user. Every message is sent as a 4-byte length followed by a JSON body like the one below. The client numbers each request with seq, the server echoes that number back in its reply, and recipients use it to check that messages from each sender arrive in order.

```
{ "type": "MSG", "session": "<token>", "seq": 42, "payload": { "room": "general", "text": "hello" } }
```

Message types

Type	Direction	Payload fields	Req.
REGISTER	Client → Server	username, password	F-001
LOGIN	Client → Server	username, password	F-002
CREATE_ROOM	Client → Server	room	F-005
JOIN / LEAVE	Client → Server	room	F-006
MSG	Client → Server	room or to, text	F-007
HISTORY	Client → Server	room or with, before_ts, limit	F-012
ADMIN	Client → Server	action, target, args	F-015, F-016
LOGOUT	Client → Server	—	F-018
PING / PONG	Both	—	F-014, SR-006
ACK	Server → Client	ref_seq, status, data	—
ERROR	Server → Client	ref_seq, code, reason	F-009, F-010
EVENT	Server → Client	kind (msg presence join leave room), data	F-004, F-007, F-013

Error codes

Code	Meaning
INVALID_MESSAGE	Frame too large, not valid JSON, unknown type, or field fails validation
AUTH_FAILED	Wrong credentials or a blocked account (kept the same on purpose so usernames cannot be guessed)
SESSION_EXPIRED	Token missing, invalid or idle beyond timeout
FORBIDDEN	Authenticated but not authorised (not a room member, not an admin)
DUPLICATE	Username or room name already exists
NOT_FOUND	Room or user does not exist
SERVER_BUSY	Connection cap reached

Connection states

State	Allowed message types	Transition
UNAUTHENTICATED	REGISTER, LOGIN, PING	→ AUTHENTICATED on successful LOGIN
AUTHENTICATED	All client types (ADMIN only for admin role)	→ CLOSING on LOGOUT, idle timeout, heartbeat loss, socket error or admin restriction
CLOSING	None	Session invalidated, presence removed, socket closed → CLOSED

Module layout (repository)

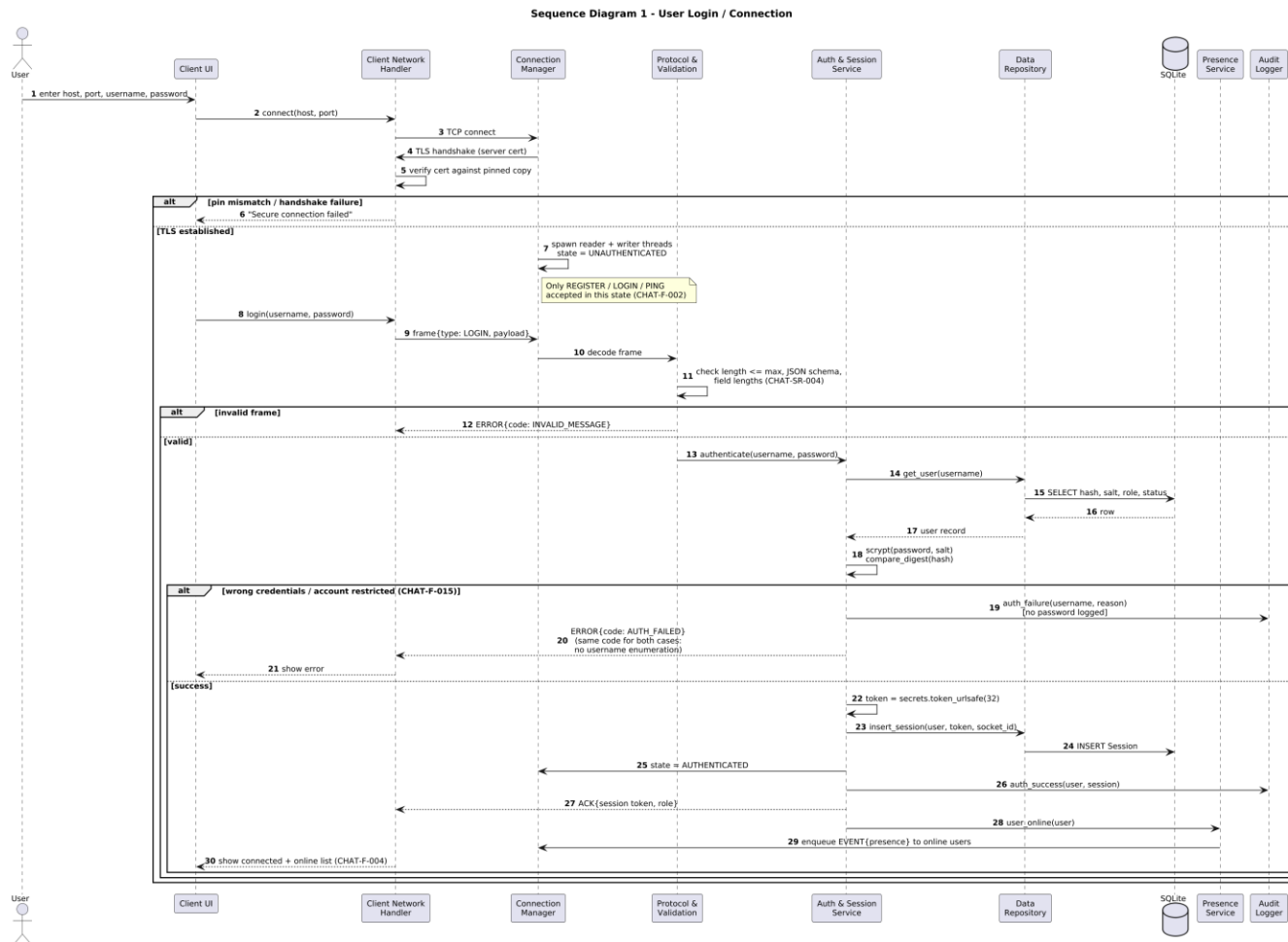
Component	Package path
Connection Manager	server/transport/connection_manager.py
Protocol & Validation	server/protocol/framing.py, server/protocol/validator.py
Auth & Session Service	server/services/auth.py, server/services/sessions.py
Presence Service	server/services/presence.py

Component	Package path
Room Manager	server/services/rooms.py
Message Router	server/services/router.py
Admin Service	server/services/admin.py
Data Repository	server/persistence/repository.py, server/persistence/schema.sql
Audit Logger	server/audit/audit_logger.py
Client UI / Client Network Handler	client/ui/, client/net/
Tests / diagrams	tests/, docs/diagrams/

4.2 UML Sequence Diagrams

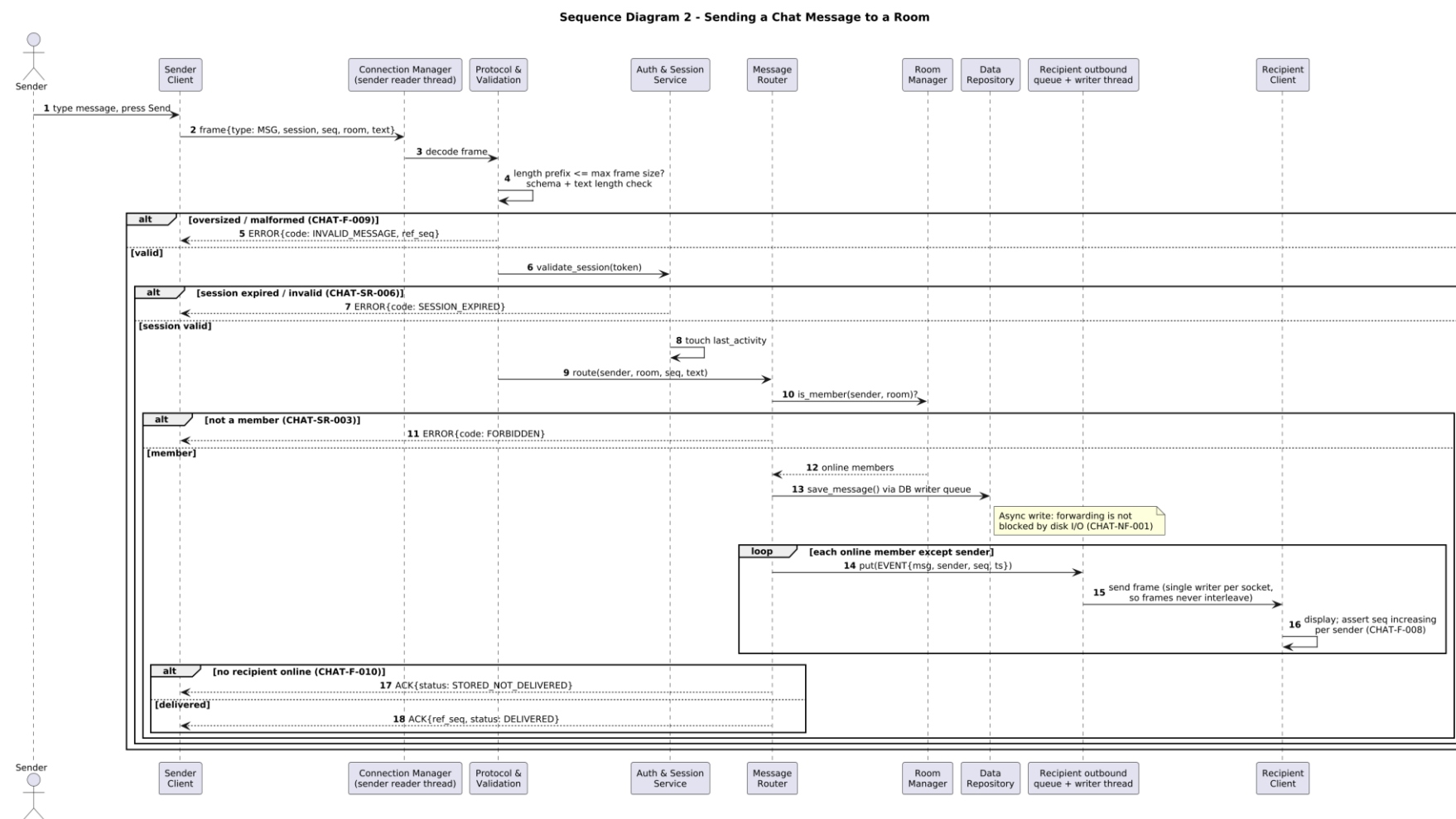
Sequence Diagram 1: User Login / Connection

The connection is set up and encrypted before the user logs in, but until login succeeds the server only accepts register, login and ping messages, so nothing else can happen on an unauthenticated connection. Whether the password is wrong or the account is blocked, the user sees the same “login failed” message, so nobody can use it to find out which usernames exist. The real reason only goes into the audit log, and passwords are never logged.



Sequence Diagram 2: Sending a Chat Message

Every message is checked first, then the session, then whether the sender is actually in the room. The message is saved in the background and sent straight away to everyone online in the room, so writing to the database never slows delivery down. Each connection has a single thread writing to it, so messages never get jumbled, and the receiving client checks that each sender's messages arrive in order.



4.3 API Design

The chat application uses a socket-based client-server interface rather than a REST API. Messages are exchanged as UTF-8 JSON bodies preceded by a 4-byte length field. The following interfaces define the main communication between the client and server components.

4.3.1 Client-Server Communication Interface

Interface	Direction	Purpose	Main Data
Client Network Handler ↔ Connection Manager	Bidirectional	Establish, maintain and close the encrypted client connection.	TCP/TLS connection, framed JSON messages
Client ↔ Protocol & Validation	Client → Server	Send requests for validation and processing.	Message type, fields, session information, sequence number
Server → Client	Server → Client	Return results, errors and asynchronous events.	ACK, ERROR, EVENT messages

4.3.2 Authentication Interface

Message	Direction	Request Fields	Response / Result
REGISTER	Client → Server	username, password	ACK or ERROR
LOGIN	Client → Server	username, password	Authenticated session or AUTH_FAILED
LOGOUT	Client → Server	No payload	Session invalidation and connection cleanup
PING / PONG	Both	No payload	Connection/session liveness check

Authentication requests are processed by the Auth & Session Service. Passwords are handled only for authentication and are not written to logs. The server deliberately uses the same AUTH_FAILED response for incorrect credentials and blocked accounts to avoid exposing whether a username exists.

4.3.3 Message / Chat Interface

Message	Direction	Request Fields	Purpose
CREATE_ROOM	Client → Server	room	Create a unique chat room
JOIN / LEAVE	Client → Server	room	Join or leave a chat room
MSG	Client → Server	room or recipient, text, seq	Send a chat message
HISTORY	Client → Server	room or recipient, before_ts, limit	Retrieve permitted chat history
ADMIN	Client → Server	action, target, args	Perform an authorized administrative operation
ACK	Server → Client	ref_seq, status, data	Confirm request processing
ERROR	Server → Client	ref_seq, code, reason	Report a rejected request
EVENT	Server → Client	kind, data	Deliver message, presence, join, leave or room events

Every incoming message is validated before the corresponding service processes it. For chat messages, the server checks the session and the sender's room membership before routing the message.

4.4 Error Handling, Logging & Monitoring

The system uses standardized error codes so that clients receive predictable responses without exposing unnecessary internal information.

Error Code	Meaning	Handling
INVALID_MESSAGE	Frame too large, invalid JSON, unknown type or invalid field.	Reject request safely before service processing.
AUTH_FAILED	Incorrect credentials or blocked account.	Use the same response to reduce username enumeration.
SESSION_EXPIRED	Missing, invalid or expired session token.	Reject request and require a new authenticated session.

FORBIDDEN	Authenticated user is not authorized.	Reject the protected operation.
DUPLICATE	Username or room name already exists.	Reject duplicate creation.
NOT_FOUND	Requested room or user does not exist.	Return standardized error.
SERVER_BUSY	Connection limit has been reached.	Reject the excess connection and close it.

Security-relevant events are recorded by the Audit Logger. Passwords and session tokens are redacted before logging. The log records who performed an action, when it occurred, what action was attempted and the outcome.

Monitoring focuses on connection failures, authentication failures, invalid requests, administrative actions, session expiry, server errors and connection capacity. Rotating logs are used so that operational logging remains manageable.

4.5 UX Design

The client uses a Tkinter desktop interface containing sign-up and login screens, a room list, online-user information, the chat area, chat history and a connection status bar.

- Authentication failures are shown using a generic login-failed message rather than revealing whether a username exists.
- Expired or invalid sessions are reported using a clear session-expired status and require the user to authenticate again.
- Unauthorized actions display a clear permission error without exposing server internals.
- Connection loss and server availability are reflected in the client status information.
- The chat interface visually distinguishes the user's own messages from messages received from other users.
- Network events are passed to the UI through a queue so that the Tkinter main thread remains responsible for UI updates.

4.6 Open Issues & Next Steps

The implementation has not yet started, so the main open issue is completing the design in the repository without deviating from the approved requirements.

- Implement connection establishment and authentication first.
- Implement message validation, routing and delivery next.

- Implement room management and chat history persistence.
- Implement administrative operations and protected audit logging.
- Add and execute the security test cases TC-SEC-01 through TC-SEC-07 from the Software Test Plan.
- Verify the 50-client scalability target and the message delivery latency requirement during testing.
- Document and update the SAD if implementation decisions differ from the current design.

The single-server architecture remains a known project limitation: if the central server goes down, chat service becomes unavailable. For the current project scope this is an accepted architectural constraint.

5. Appendices

5.1 Glossary

Term	Definition
Socket	A network endpoint used for client-server communication.
TCP	Transmission Control Protocol used to provide reliable, ordered communication.
TLS	Transport Layer Security used to encrypt communication between client and server.
Certificate Pinning	A mechanism where the client accepts only the configured server certificate.
Session Token	A random value issued after successful login to identify an authenticated session.
RBAC	Role-Based Access Control used to restrict operations according to user roles.

scrypt	A password hashing function used with a unique random salt for each user.
STRIDE	A threat-modeling framework covering spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege.
Audit Log	A protected record of important security and operational events.
Frame	A network message consisting of a 4-byte length followed by a JSON body.

5.2 References

- Software Requirements Specification (SRS), Chat Application using Socket Programming, v1.0, 04-09-2026.
- Software Test Plan (STP), Chat Application using Socket Programming, v1.0, 28-09-2026.
- Requirements Traceability Matrix (RTM) associated with the SRS.
- IEEE 42010 – Architecture Description standard.
- OWASP security guidance.
- NIST SP 800-160 – Systems Security Engineering.

5.3 Tools

Tool / Technology	Purpose
Python 3.11+	Implementation language and runtime for the client and server.
socket / ssl / struct	TCP networking, TLS protection and message framing.
threading / queue	Connection handling and communication between network and UI threads.
sqlite3	Persistent storage using SQLite and WAL mode.

hashlib.scrypt / hmac / secrets

logging / RotatingFileHandler

Tkinter

pytest

PlantUML

Password hashing, secure comparison and session-token generation.

Protected and rotating server/audit logs.

Desktop client user interface.

Automated testing.

Architecture and UML sequence diagrams.